Broader Semantics



COMS4113

COMS4113: Fundamentals of Distributed Systems

Instructor: Hubertus Franke

Columbia University

https://frankeh.github.io/cu-ds1-2026-fall

Broader Semantics

A broader view of isolation and consistency

Context

- For four lectures we built toward **strong semantics** - a distributed system that behaves like a non-distributed, non-concurrent one:
  - **Serializable isolation** - transactions appear to execute sequentially; their individual operations do not interfere.
  - **Linearizable consistency** - across all replicas, operations appear to execute in an order consistent with the *real-time* ordering of when they completed.
- **Spanner achieves both, and pays for them** - in latency, in coordination, and in atomic clocks.
- **That price is not always worth paying.** First, some real systems that chose not to pay it. Then the well-understood weaker semantics: what each permits, and what each saves.

Who Runs on Weak Guarantees?

You already use all of these. None of them is serializable or linearizable - and in every case that was a *decision*, not an accident.

System	What a user could notice	What it gives up
DNS	the old address, until cached copies expire	a change reaches caches one by one, not at once
Facebook (TAO)	a like or comment count a moment behind	replicas catch up; they are not updated in lockstep
Amazon cart	an item you deleted comes back	always accepts a write, even during failures; merges later
Bank ATM	cash from a balance it cannot fully check	checks a limit instead, and settles up afterwards
Amazon S3	a file you just saved missing from a listing	was eventually consistent - until December 2020

Next: how each one works - and the one question that decides whether giving this up is acceptable.

Case 1: DNS

authoritative server

the owner of example.com

`www = 5.6.7.8`

click 1: the owner moves the site

resolver	its answer	its copy expires in *(illustrative)*
cache A	`5.6.7.8`	refreshed
cache B	`5.6.7.8`	refreshed
cache C	`5.6.7.8`	refreshed
cache D	`5.6.7.8`	refreshed

A cache only asks again when its own copy expires.

Why nobody is hurt

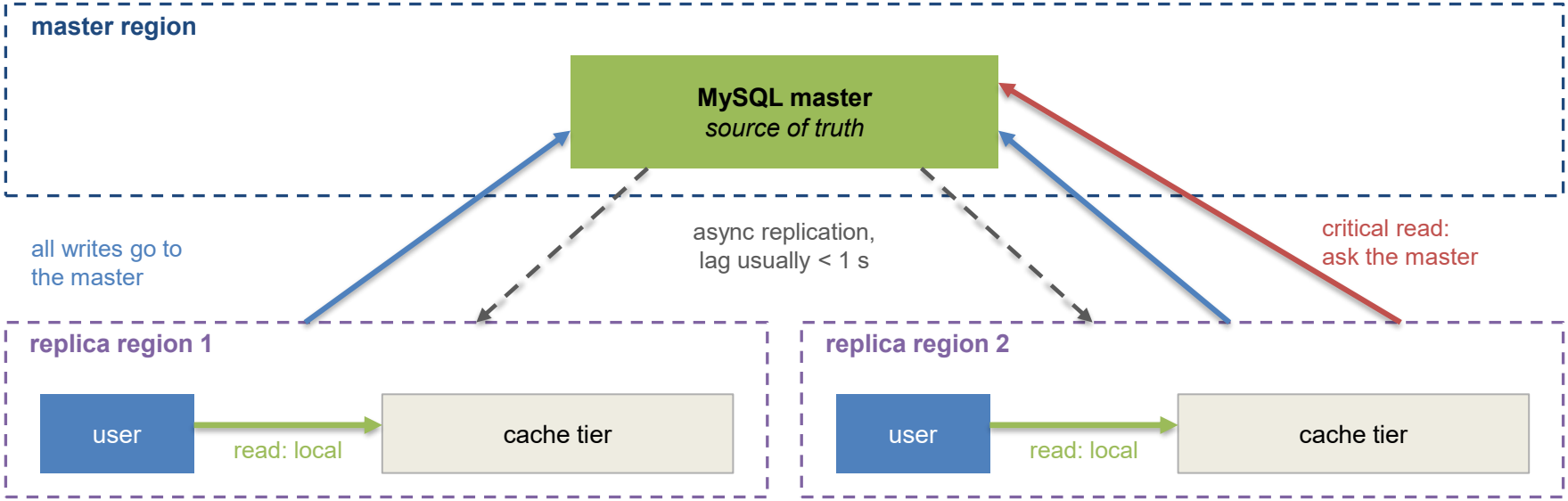
Names change rarely, so a stale answer is the exception - and the owner can plan for it.

It is deliberate

RFC 1034: the TTL "can be reduced prior to the change to minimize inconsistency during the change".

Vogels: DNS is "the most popular system that implements eventual consistency". TTLs shown are illustrative; RFC 1034 s3.6.

Case 2: A Social Graph at Scale



Why it is OK. The paper: the application "should not expect the data to be stale in the common case, but should be able to tolerate it". A count a second old harms no one.

And it can ask for more. A *critical read* - their example is authentication - goes to the master, "so that replication lag doesn't allow use of stale credentials".

Bronson et al., TAO, USENIX ATC 2013: "a billion reads and millions of writes each second"; "explicitly favors availability".

Accept Now, Repair Later

Amazon cart *Dynamo, 2007*	**Accepts:** "add to cart" always works, even while servers fail	**Mistake:** two copies of the cart are merged - a deleted item can come back	**Repair:** you delete it again. A click.
Bank ATM *Brewer, 2012*	**Accepts:** network down, so it pays out - up to a limit (say $200)	**Mistake:** the true balance may already have gone below zero	**Repair:** an overdraft fee, an audit. Loss capped by the limit.
Coffee shop *Hohpe, 2005*	**Accepts:** takes the order and the money, queues the cup	**Mistake:** the drink is wrong, or the machine breaks	**Repair:** remake it, or refund the money.

The pattern: say yes now, detect and fix afterwards. A two-phase commit (lecture 8) would make the customer stand at the till, money in hand, until the drink is ready. The business keeps the sale and fixes the rare mistake.

Dynamo s2.3 and s4.4; Brewer 2012; Hohpe 2005. The $200 limit is Brewer's example figure.

Why Is That OK?

1. How bad is it?

A like count or a DNS record: nobody is hurt. A balance you are about to spend: someone is.

2. Can it be fixed?

Remove an item, remake a drink, charge a fee. Some mistakes cannot be undone.

3. What does strict cost?

A cross-world round trip on every read, or an error screen when a link fails.

	can be fixed afterwards	cannot be fixed
hurts little	**Weak is fine.** DNS, like counts, a cart, a coffee	**Weak is still fine.** Little harm, so no repair is needed
could hurt a lot	**Weak + a safety net.** ATM: a limit, then a fee and an audit	**Strong needed.** Where Spanner lives. Two primaries; a debit with no credit

Why is an ATM allowed to do this?

We said a bank needs consistency. What it really needs is that **money is never lost or invented in the end** - not that every ATM sees the exact balance at every instant.
The ATM caps the damage (a limit). **The bank repairs the rest** (a fee, an audit).
Refusing every customer whenever a link fails costs sales - Brewer: *"higher availability means higher revenue."*

Brewer 2012: in stand-in mode the ATM does not know the true balance; the limit bounds the risk, the audit repairs it.

The Trade Can Move: Amazon S3

2006 to 1 Dec 2020: eventually consistent		1 Dec 2020 on: strongly consistent
What you could see After a PUT, "a small time window where the data has been accepted and durably stored, but not yet visible to all GET or LIST requests."	**What customers built** Workarounds: EMRFS Consistent View, S3Guard, custom code, or extra databases to keep objects in step.	**What you get now** "What you write is what you will read." Every GET, PUT and LIST, in all regions.

Why did AWS change it?

Demand moved: big-data analytics increasingly need an object *immediately after* it is written.
The workarounds had a price: custom code or extra databases. Dropbox moved a 34 PB data lake and could simplify.
Strictness became cheap: AWS says no extra charge, no impact on performance, no global dependencies.

Same three questions, new answers: a stale read got *more* costly; a strict one got *cheaper*.

AWS News Blog (J. Barr) and AWS What's New, 1 Dec 2020.

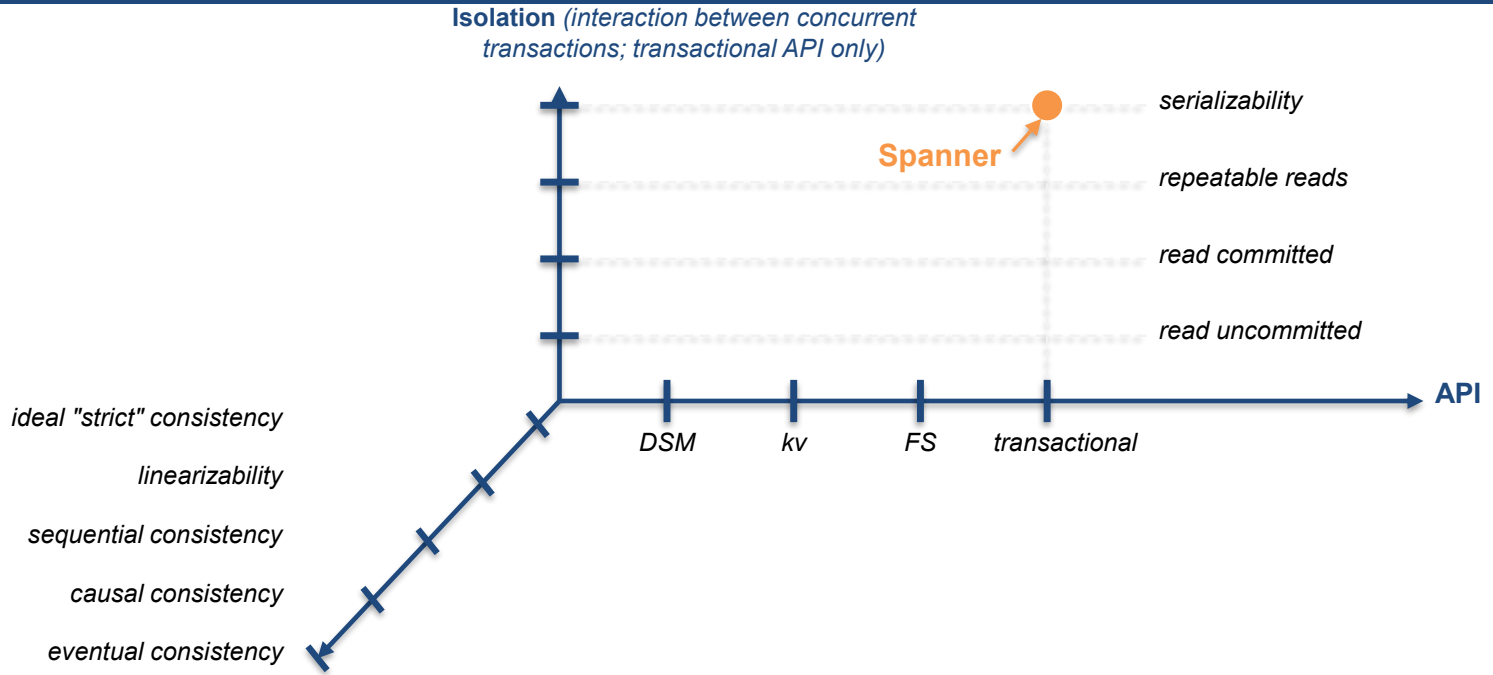
What Went Wrong, Exactly?

Every example above is a *named* anomaly - and the rest of the lecture defines the models that rule each one out.

Example	What the user saw	What went wrong	What rules it out
DNS	the old address after a change	a stale read	linearizability (slide 27)
TAO	a count a second behind	a stale read	linearizability (slide 27)
Cart	a deleted item came back	concurrent writes had to be merged	not ruled out - the price of eventual consistency (slide 38)
ATM	two withdrawals each passed the check	decisions fine alone, rule broken together	serializability; write skew (slide 18) has the same shape
S3, before 2020	a just-saved file missing from a list	a read did not see its own write	linearizability, or just read-your-writes (slide 40)
Photo album	an album listing a photo you cannot open	an effect seen before its cause	causal consistency (slide 32)

A thought experiment from the COPS paper (Lloyd et al., SOSP 2011), not a measured system.

The Design Space



The API axis, weakest contract to richest: DSM = *distributed shared memory,* `read(x)`/`write(x)` **kv** = *key/value,* `get`/`put` **FS** = *file system* **transactional** = *`BEGIN...COMMIT`*
Spanner: transactional, strongest isolation.

Third axis: *consistency is a* **separate** *choice.*

Isolation Semantics

a.k.a. isolation levels

Isolation Semantics

- **Relevant only for transactional APIs.**
- Define how concurrent transactions interact - whether the individual effects of an in-progress transaction can be *witnessed* by another transaction.
- The gold standard is **serializability**: transactions are completely isolated from one another.
  - Defined as: **any history of execution is equivalent to some serial execution.**
  - To achieve it, the engine must serialize conflicting transactions - which is expensive.
- **Weaker levels exist.** They permit specific *anomalies* in exchange for concurrency: more anomalies, more corner cases for the programmer, better performance.

Best Known Isolation Levels

Serializability

Repeatable reads

Read committed

Read uncommitted



Better
performance



Worse
programmability

A useful mnemonic: read down the list as *locks you stop taking, or stop holding as long*. Each step down releases something, buys concurrency, and admits one more anomaly.
Lockless implementations also exist - and a few slides on, snapshot isolation breaks this mnemonic entirely.

What Locks Are We Talking About?

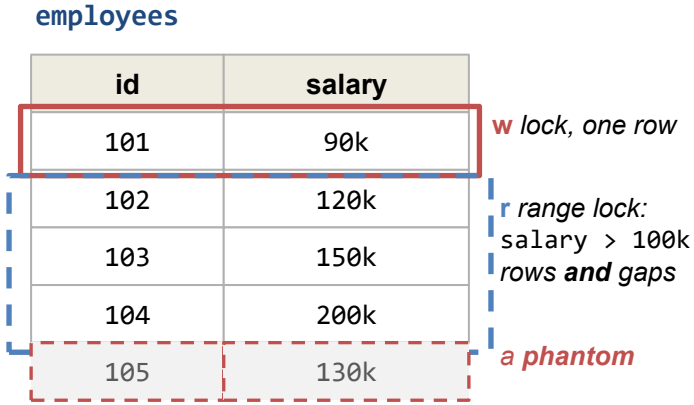
Lecture 7 gave us row-level locks and how long to hold them. The isolation levels need **two more distinctions**, and neither has appeared yet:

1. Mode: r vs w

A **w** (exclusive) lock blocks everyone. An **r** (shared) lock blocks only writers. *Reads share; writes do not.*

2. Granularity: row vs range

A **row** lock protects one existing row. A **range** (predicate) lock protects every row matching a *condition*, **including the gaps** where a row does not exist yet.



Why the gaps matter: a row lock cannot stop an *insert*. Only a range lock can - and that is the whole difference between the top two levels.

Brief Descriptions

Serializability

Take **r/w row** locks *and* **r range** locks; hold them for the entire transaction.
Ensures all conflicting, concurrent transactions are isolated from each other.
Anomalies: **none.**

Repeatable reads

Take **r/w row** locks, hold for the entire transaction. Take **no range locks at all.**
Ensures every row-level read is repeatable.
Anomaly: **phantom reads** - a concurrent transaction adds or removes a row matching another transaction's *range* query.

Brief Descriptions (cont.)

Read committed

Take write row-level locks, hold for the entire transaction. Take read row-level locks but hold them **only while the row is being read.** No range locks.
Ensures only committed updates are read.
Anomalies: phantom reads **+ non-repeatable reads** - read a row twice, get two answers.

Read uncommitted

Take write row-level locks, hold for the entire transaction. **No read locks at all**, row-level or range.
Ensures only that rows are written atomically.
Anomalies: phantom + non-repeatable **+ dirty reads** - read a write that may yet abort.

Snapshot Isolation

The most widely deployed non-serializable level is not on that list. Rather than locking reads, give each transaction a *consistent snapshot* as of its start time; on a write conflict, **first committer wins** and the loser aborts.

What it prevents

No dirty reads, no non-repeatable reads - and unlike the deck's repeatable read, **no phantom reads either.** Reads never block writes; writes never block reads.

What it permits: write skew

Two transactions read the same rows, then each writes a *different* row. Neither write conflicts, both commit, and a constraint spanning both rows breaks.

Write skew, concretely: the invariant is `x + y >= 0`, with `x = y = 50`. T1 reads both, sets `y = -40`. T2 reads both, sets `x = -40`. Both commit. Now `x + y = -80`, and **no anomaly above describes it.**

So the ladder picture breaks

a common conception that repeatable read and snapshot isolation are *incomparable*. Snapshot Isolation forbids phantoms but permits write skew; repeatable read permits phantoms but forbids write skew. **Neither one is stronger** - the levels form a lattice, not a line.

Comparisons

- Anomalies make it progressively harder for the programmer to reason about what the database will do.
- But less synchronization means better performance - and that holds for lockless implementations too.
- **Commercial databases mostly default to a weak level, not to serializability.** Read committed is the most common default - but it is *not* universal, and the level **names are not portable**. The next slide is the verified table.
- You can raise the level per transaction for the ones that need it, and the semantics compose.

What Databases Actually Default To



Engine	Default level	Worth knowing
PostgreSQL	Read committed	`REPEATABLE READ` *is* snapshot isolation - and forbids phantoms
Oracle	Read committed	Offers **only** read committed, serializable, read-only
SQL Server	Read committed	Azure SQL defaults to read-committed **snapshot** (row versioning)
MySQL / InnoDB	Repeatable read	**Not read committed**

The lesson is the right-hand column, not the middle one. The same level *name* means different things in different engines: PostgreSQL's `REPEATABLE READ` is snapshot isolation and forbids phantoms, where the standard says it permits them. Oracle has no `REPEATABLE READ` at all.

So never reason about isolation from the level name alone - read your engine's docs. And note that `SERIALIZABLE` in Oracle is snapshot isolation, not serializability.

Consistency Semantics

a.k.a. consistency models

Consistency Semantics

- **Relevant to transactional and non-transactional APIs alike.** We will work in the non-transactional setting, where each operation is a single read or write.
- Constrain the *order* in which individual operations are **witnessed** by different readers.
- The gold standard is **linearizability**: operations are seen in the real-time order in which they completed. Achieving it means coordinating replicas, waiting out clock uncertainty - all expensive, as lecture 9 showed in detail.
- **Weaker models exist**, buying performance, scalability and sometimes *availability* - at the price of more corner cases. The last new slide of this lecture argues the price is usually paid in **latency**, not availability.

Best Known Consistency Models

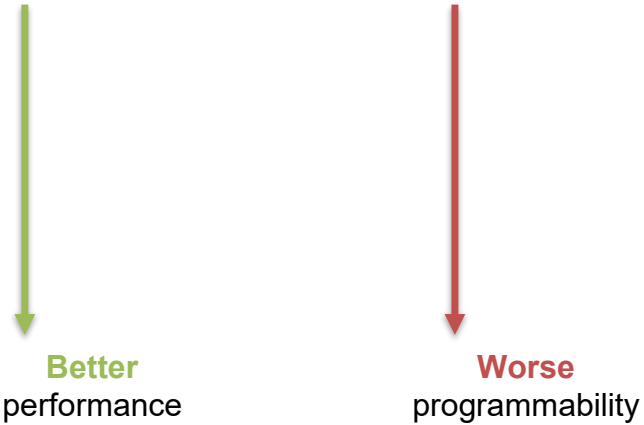
Ideal "strict" consistency

Linearizability

Sequential consistency

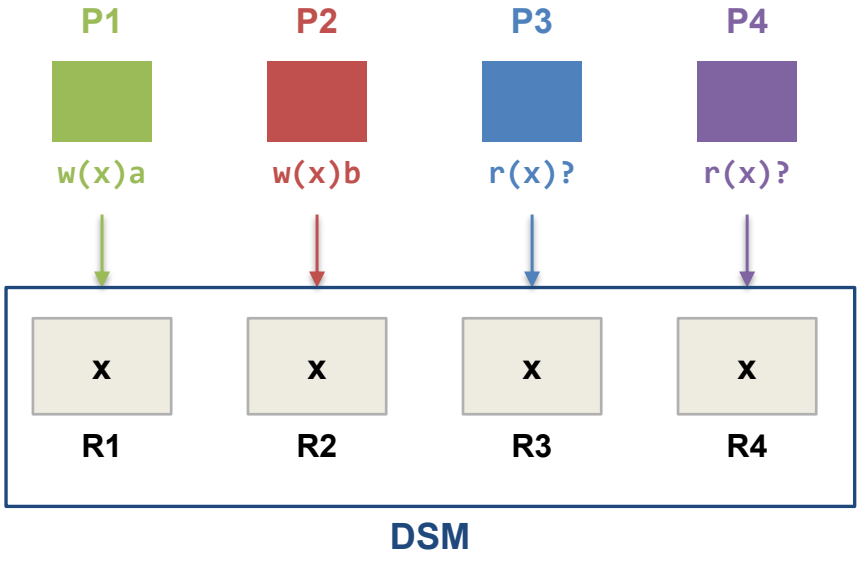
Causal consistency

Eventual consistency



Every variation reduces to two questions: (1) how *stale* may a read be, and (2) how much do replicas have to agree on the *order of writes*?
Keep those two questions in mind for the next ten slides - each model is a different answer to them.

Replicated Shared Memory



x = nil initially, at every replica

To define consistency we must say **which values a read is allowed to return.**

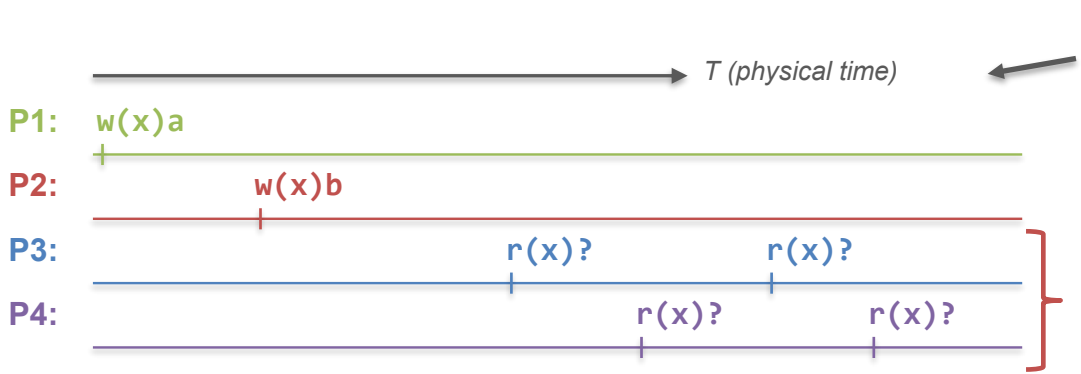
A consistency model restricts *all possible executions* - not just the one you happened to observe.

So each example that follows asks: **is this particular execution admissible** under the model, or not?

One bad execution disproves a model. No number of good ones proves it - which is what lecture 14 is about.

Why a DSM and not a database? Single reads and writes separate the *ordering* question from the *grouping* question - everything that follows is about which value a read may return. *For a transactional API, read "operation" as "transaction" throughout.*

Structure of an Example



Time at which the **client process issues** the operation.

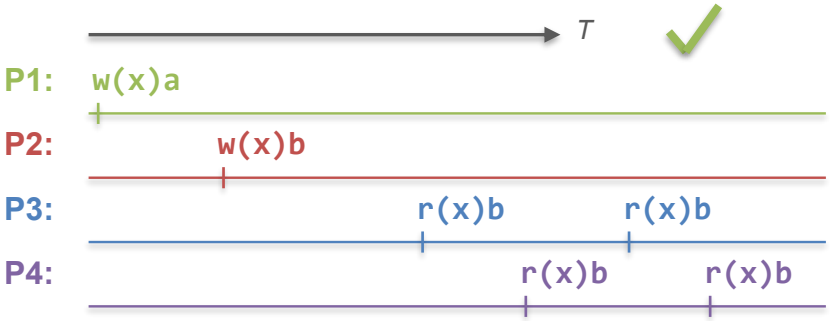
May differ from the time the request actually reaches the relevant replica!

How to read these diagrams. One row per client process, left to right in physical time. `w(x)a` writes value a to x; `r(x)b` is a read that *returned* b; `r(x)?` is the question being asked.
P1 and P2 write. P3 and P4 read. The two writes here are *concurrent* - neither finished before the other began.

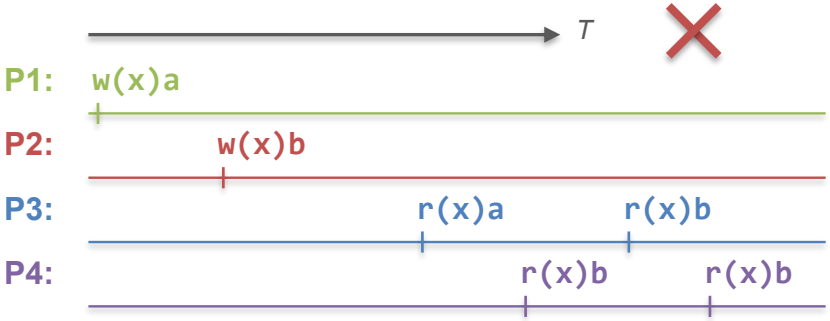
So: what may those four reads return? Every model in this lecture answers that question differently - and this one execution is the question each of them is asked.

Strict Consistency

Defn: any execution is the same as if all read/write operations were executed in the order of the **physical time at which they were issued.**
Therefore: (1) reads are never stale; (2) every replica enforces physical-time ordering for all writes.



Admissible. Both readers see `b` as soon as it is written - no read is ever stale.

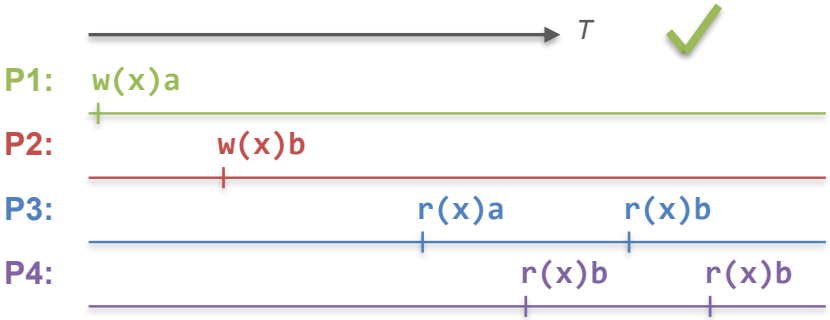


Not admissible. P3's first read returns `a`, although `w(x)b` had already been *issued*.

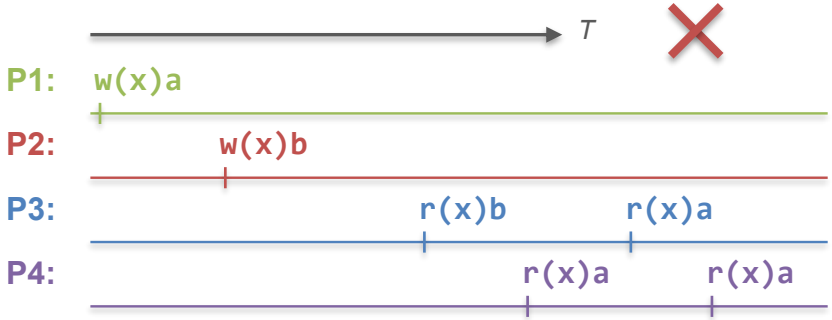
*Keep the right-hand execution in mind: linearizability will **accept** it, and why is the next slide.*

Linearizability

Defn: any execution is equivalent to one in which every operation took effect **instantaneously at some instant between when it was issued and when it returned** - so the order **respects real time** for non-overlapping operations, and each read returns the most recent completed write.
Therefore: (1) once a write completes, all later reads return it or a later value; (2) once a read returns a value, no later read returns an older one.



Admissible - though *not* strictly consistent. P3's `r(x)a` **overlaps** `w(x)b`, so there is a legal instant for it.



Not admissible. P3 returns `b`, then `a` - a read went **backwards**, which rule (2) forbids.

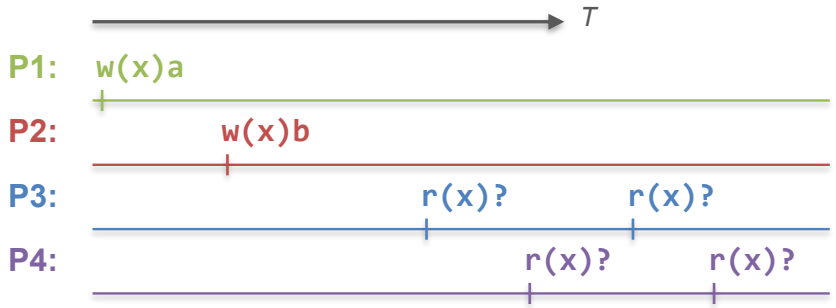
Strict consistency rejected the left-hand execution; linearizability accepts it - the gap between *issue* time and *completion* time.

Implementability

- **The ideal is not implementable; linearizability is.** Strict consistency needs a global physical clock read instantly and exactly by every replica. No distributed system has one.
- **Spanner guarantees linearizability** for its transactions - treat each transaction as one operation - using locks, commit wait, and the protocol combination from last lecture.
- So Spanner's semantic is precisely **serializable isolation + linearizable consistency**, one choice on each of our two axes:
  - Traditionally that pairing is called **strict serializability**.
  - The Spanner paper calls it **external consistency**.
- **But linearizability is not only about transactions** - it applies to DSMs, key/value stores and file systems just as well, which is why we define it on plain reads and writes.

Sequential Consistency

Defn: any execution is the same as if all read/write operations were executed in **some global order**, and the operations of each client process appear in that order **as specified by its program.**
(A global order that respects program order is called a **global sequential order***.)*
Therefore: (1) reads may be stale in real time, but not in *logical* time; (2) writes are totally ordered, consistently, across all replicas.

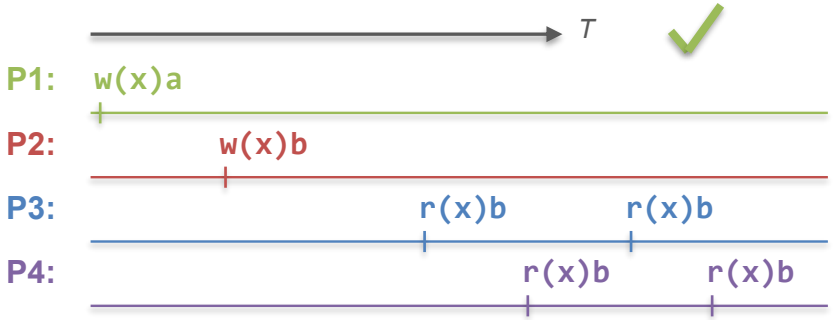


If the DSM were sequentially consistent, what may these reads return?

The one word that changed from linearizability: *real time* became *some order*. There is still a single global order every replica agrees on - it simply no longer has to match the wall clock. *That is the entire difference, and it is worth a lot of performance.*

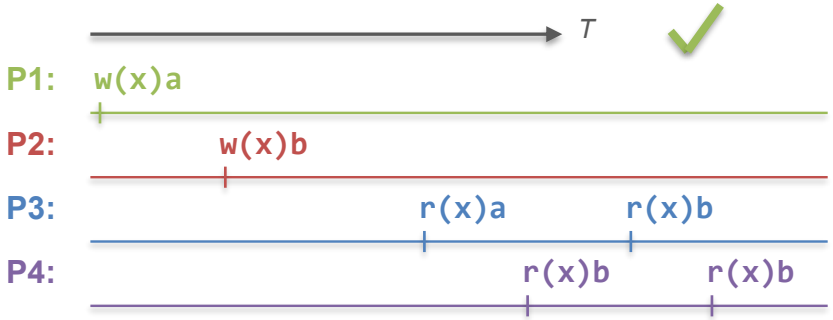
Sequential Consistency: Admissible

Defn: some **global order**, respecting each process's **program order**. Reads may be stale in real time - but every replica must agree on one order.



A global order that explains it:
physical-time order works -
`w(x)a, w(x)b, r(x)b, r(x)b, ...`

Admissible - and this one is **strictly consistent** too.

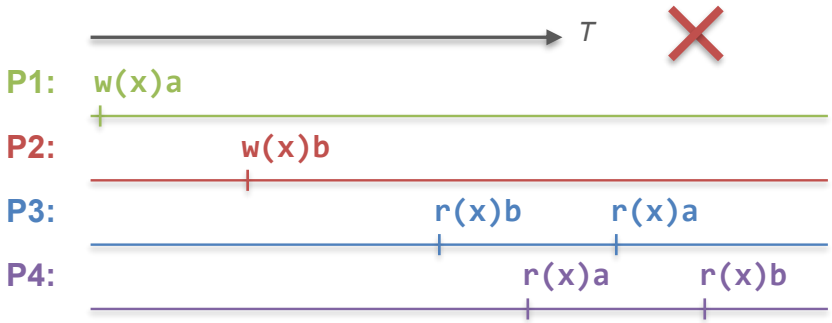


A global order that explains it:
`w(x)a, r(x)a, w(x)b, r(x)b, ...`
P3's stale read is simply placed earlier.

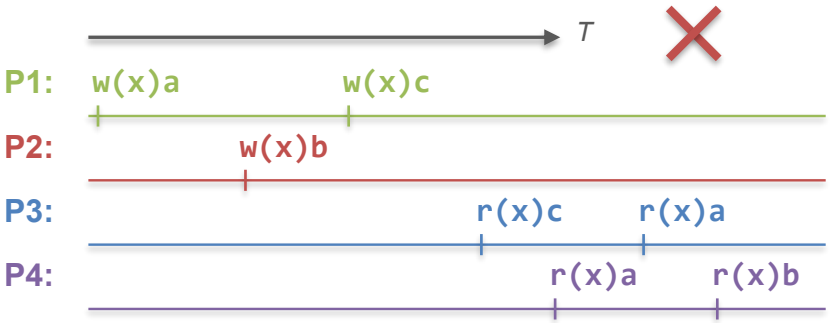
Admissible - and **linearizable**, but *not* strictly consistent.

Sequential Consistency: Refuted

Defn: some **global order**, respecting each process's **program order**.



No global order can explain this.
P3 sees `b` then `a`; P4 sees `a` then `b`. A single order would have to put `w(x)a` both before *and* after `w(x)b`.
=> not sequentially consistent.



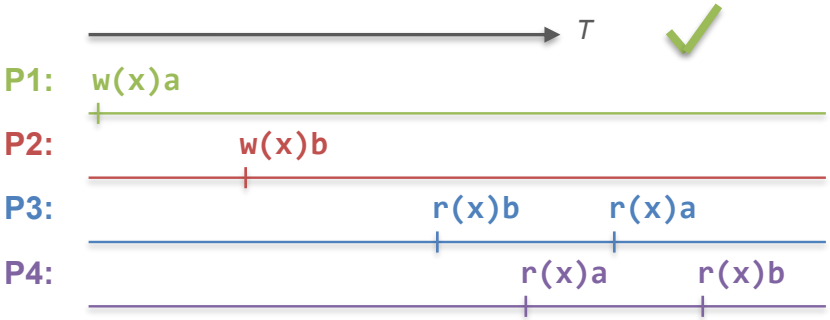
Nor this one - and for a different reason.
P3 reads `c` then `a`. But P1's *program order* is `w(x)a` then `w(x)c`.
Any order explaining P3 would need `w(x)c` before `w(x)a`, **violating P1's program order.**

Causal Consistency

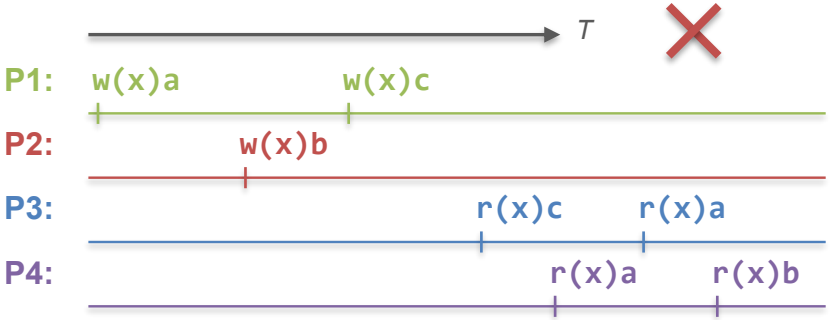
- **Defn:** any execution is the same as if all **causally-related** read/write operations were executed in an order that **reflects their causality.**
  - **Concurrent** operations - those with no causal path between them - **may be seen in different orders by different processes.**
- **Therefore:** (1) a read is fresh only with respect to the writes it causally depends on; (2) only causally-related writes are ordered identically at every replica.
  - Concurrent writes may be committed in different orders by different replicas - and so read in different orders by different applications.
- **This is the first model that gives up the single global order.** Everything above it in the list had one; causal consistency keeps only the *partial* order that causality forces.

Causal Consistency: Concurrent

Defn: causally-related operations must be seen in causal order. **Concurrent operations may be seen in different orders by different processes.**



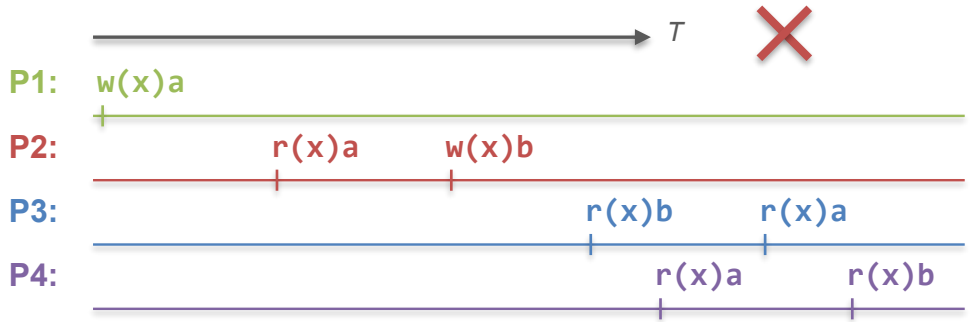
Admissible. The only ordering constraints are per-process: `w(x)b` < `r(x)b`; `r(x)b` < `r(x)a`; ...
`w(x)a` || `w(x)b` - they are *concurrent*, so P3 and P4 may disagree about their order.



Not admissible. Having read `c`, P3 must keep reading `c` or something newer - it cannot go back to `a`, because `w(x)c` was issued *after* `w(x)a` completed in P1's program, so `w(x)a` → `w(x)c` causally.

Causality Through a Read

Defn: causally-related operations must be seen in causal order. *Causality also flows through a read.*



The causal chain: P2's `w(x)b` follows P2's `r(x)a`, which read P1's `w(x)a`. So `w(x)a` → `r(x)a` → `w(x)b`, and by transitivity `w(x)a` → `w(x)b`.

So this execution is not admissible. The system must enforce `w(x)a` before `w(x)b` everywhere - but **P3 reads b, then a**, inverting them.
Note the contrast with the previous slide: there the two writes were concurrent and disagreement was legal. Here a single read created the dependency.

Why Causal Consistency?

- Causal consistency is **strictly weaker** than sequential consistency, and it produces results that look strange - as you have just seen.
- **But it requires no cross-replica agreement**, and therefore no coordination round trip. That is a large, structural performance win, not a constant factor.
- In practice, causally consistent systems do **not** invert concurrent updates to the *same object* - preventing that is cheap and easy.
  - The visible weirdness is across *different* objects: `w(x)5` concurrent with `w(y)7` may be seen in either order by different replicas.
- **It is also the strongest model that stays available during a network partition** - which is why it keeps appearing in systems that must not stop.

The Same Executions, Four Verdicts

Every example in this lecture used one of the same few executions. **P1 and P2 each write once; P3 and P4 each read twice.** Collected in one place, the pattern is the point:

P3 reads	P4 reads	Strict	Linearizable	Sequential	Causal
`r(x)b r(x)b`	`r(x)b r(x)b`	✓	✓	✓	✓
`r(x)a r(x)b`	`r(x)b r(x)b`	✗	✓	✓	✓
`r(x)b r(x)a`	`r(x)a r(x)a`	✗	✗	✓	✓
`r(x)b r(x)a`	`r(x)a r(x)b`	✗	✗	✗	✓
`r(x)c r(x)a`	`r(x)a r(x)b`	✗	✗	✗	✗

Row 5's P1 issues `w(x)a` *then* `w(x)c`*, so program order is at stake; every other row has P1 and P2 writing one concurrent value each.*

Read down any column: each model admits everything the stronger one did, and more. That staircase *is* the containment hierarchy - strict ⊂ linearizable ⊂ sequential ⊂ causal ⊂ eventual.

Eventual Consistency

- Allow stale reads, but guarantee that reads **eventually** reflect previously written values - possibly after a long time.
- **Does not order writes as they execute**, which can create **conflicts** later: which write was "first"?
- Used by **Amazon's Dynamo**, the key/value store that made the model famous - and by a great deal else:
  - Many academic systems, and file synchronization.
  - Source control - **git** is an eventually consistent replicated store with an explicit, manual conflict resolver.
- **Today the boundary is a per-request flag, not a system-wide choice.** DynamoDB, Dynamo's descendant, offers *eventually consistent* reads by default **and** *strongly consistent* reads on request - `ConsistentRead` on the same API.

Sequential vs. Eventual Consistency

- **Sequential: pessimistic** handling of updates.
  - Decide the update order *as the updates execute*. Requires coordination, so it costs a round trip - but there is never a conflict to resolve.
- **Eventual: optimistic** handling of updates.
  - Let the updates happen; decide their order *later*. No coordination, so no round trip - but now conflicts are possible.
  - **And resolving them is the real cost.** It is manageable for code - you have all merged a branch - and genuinely hard in general. How do two people's edits to the same *image* merge?
- **The choice is therefore not "strong or weak".** It is *when* you pay for ordering: up front in latency, or afterwards in reconciliation logic that you have to write.

Why (Not) Eventual Consistency?

✓ **Supports** disconnected **operation and network partitions**

- Better to read a stale value than nothing at all
- Better to save a write somewhere than to lose it

✓ **Supports** increased parallelism

- Though that is rarely what it has been used for in practice

✗ **Can produce** anomalous **application behaviour**

- Stale reads and conflicting writes, exposed to your users

The honest summary: eventual consistency does not remove the need to reason about order - it *moves that reasoning into your application*, where the database can no longer help you.
Back to the opening cases: DNS, the social graph, the cart and the ATM were fine because a wrong answer was cheap to tolerate or cheap to fix. Account balances and the last seat on a flight are neither.

Other Consistency Models

- **Session guarantees** - the four properties a single client's session can expect even from a weakly consistent store *(Terry et al., 1994)*:
  - **Read your writes** - reads reflect your own previous writes.
  - **Monotonic reads** - successive reads reflect a non-decreasing set of writes.
  - **Writes follow reads** - writes propagate after the reads they depend on.
  - **Monotonic writes** - writes propagate after writes that logically precede them.
- These are *not* a footnote. "Session consistency" in a real cloud database is exactly read-your-writes plus writes-follow-reads - see the next slide.
- **In-house models** also exist: the Andrew File System's *close-to-open* consistency; the Google File System's *atomic at-most-once appends*.
- *Further reading: Tanenbaum & Van Steen §7.3, client-centric consistency models. Not required for the exam.*

Consistency as a Knob

The spectrum on slide 23 is now a product API. Azure Cosmos DB is **one service** whose consistency level is a *setting*, not five different products:

Strong	Bounded staleness	Session	Consistent prefix	Eventual

*Set as the account default; a single **read** may override it per request. Writes may not - and the classic override can only **weaken**, never strengthen.*

It selects a quorum, not a protocol

One replication protocol underneath. The level sets **how many replicas a read must consult**:
Strong - local *minority* reads, **global majority** writes.
Session / prefix / eventual - a **single replica**.

And the cost is latency

PACELC: CAP covers partitions; *else* - the normal case - you trade **latency** for **consistency**.
Strong across regions costs **2x the RTT between the farthest two**, and halves read throughput.

And note what "Session" is: *read-your-writes* + *write-follows-reads* - Terry's 1994 session guarantees, sold as a tier, and their most widely used level.

So the modern question is not "is my database consistent?" It is *"which level, for which request, and what does that one cost me?"* - which is the same question Spanner answered by buying atomic clocks.

Key Papers

- **[Berenson-1995]** Berenson, Bernstein, Gray, Melton, O'Neil, O'Neil. *A Critique of ANSI SQL Isolation Levels.* **SIGMOD 1995, pp. 1–10**, DOI 10.1145/223784.223785 - snapshot isolation, write skew, and why the lock-based and anomaly-based definitions disagree.
- **[Herlihy-Wing-1990]** *Linearizability: A Correctness Condition for Concurrent Objects.* ACM **TOPLAS 12(3):463–492**, DOI 10.1145/78969.78972 - the definition on slide 27.
- **[Lamport-1979]** *How to Make a Multiprocessor Computer That Correctly Executes Multiprocess Programs.* IEEE **Trans. Computers C-28(9):690–691**, DOI 10.1109/TC.1979.1675439 - sequential consistency.
- **[Ahamad-1995]** Ahamad, Neiger, Burns, Kohli, Hutto. *Causal memory: definitions, implementation, and programming.* **Distributed Computing 9(1):37–49**, DOI 10.1007/BF01784241.
- **[Terry-1994]** Terry et al. *Session Guarantees for Weakly Consistent Replicated Data.* **PDIS 1994, pp. 140–149**, DOI 10.1109/PDIS.1994.331722 - the four guarantees on slide 40.
- **[DeCandia-2007]** *Dynamo: Amazon's Highly Available Key-value Store.* **SOSP 2007, pp. 205–220**, DOI 10.1145/1294261.1294281; and **[Elhemali-2022]** *Amazon DynamoDB.* **USENIX ATC 2022, pp. 1037–1048** - the paper, and the product it became.
- **[Ports-Grittner-2012]** *Serializable Snapshot Isolation in PostgreSQL.* **PVLDB 5(12):1850–1861**, DOI 10.14778/2367502.2367523 - SSI, shipped in PostgreSQL 9.1.
- **[Abadi-2012]** *Consistency Tradeoffs in Modern Distributed Database System Design.* IEEE **Computer 45(2):37–42**, DOI 10.1109/MC.2012.33 - PACELC. Same issue as Brewer's *CAP Twelve Years Later*, DOI 10.1109/MC.2012.37.
- Also: **[Adya-1999]** MIT PhD thesis, implementation-independent definitions of these levels; PostgreSQL, MySQL, Oracle and SQL Server isolation documentation; Azure Cosmos DB consistency levels; AWS DynamoDB read-consistency docs.